

Radixweb Placement Preparation

2027 Batch Campus Drive

Vishal Chudasama · MCA, LJ University, Ahmedabad

Applying for: **React Developer & PHP Developer**

Company	Radixweb (radixweb.com)
Location	Ahmedabad
Roles Offered	AI/ML Developer, Python/React Developer, MERN Developer, .NET Developer, PHP Developer, Mobile Developer
Probation Period	6 Months
Stipend (during probation)	₹10,000 total — ₹5,000 paid monthly + ₹5,000 on successful completion
Salary Post-Probation	Up to 5 LPA CTC
Service Bond	2 Years (excluding training period), with ₹1.5 Lakh security deposit via cheque

This document contains 70 practice questions (30 Aptitude + 20 Technical + 20 Interview) built for revision before the Radixweb drive.

Note: the drive date and eligibility criteria were not finalized at the time this was prepared, and it is not confirmed whether a separate coding round will be held — a few basic coding questions are included as a precaution.

Contents

Section 1	Aptitude & Reasoning	30 Questions
Section 2	Common + Job-Related Technical (React / PHP / Web fundamentals)	20 Questions
Section 3	Extra Interview Preparation (Technical + HR)	20 Questions

Section 1 · Aptitude & Reasoning (30 Questions)

Fresher placement-level difficulty. Each question has 4 options, the correct answer marked, and a step-by-step solution where calculation is involved.

Percentage

Q1.

In an election between two candidates, the winner secured 60% of the total votes and won by a margin of 400 votes. What was the total number of votes polled?

- (A) 1800
- (B) 2000
- (C) 2200
- (D) 2500

Answer: (B) 2000

Solution: Winner = 60% of total, Loser = 40% of total. Margin = 20% of total = 400 votes.
So, Total = $400 \div 0.20 = 2000$.

Percentage

Q2.

The price of an item is first increased by 20% and then decreased by 20%. What is the net percentage change in price?

- (A) No change
- (B) 4% decrease
- (C) 4% increase
- (D) 2% decrease

Answer: (B) 4% decrease

Solution: Let original price = 100. After +20%: 120. After -20%: $120 \times 0.80 = 96$.
Net change = $96 - 100 = -4$, i.e. a **4% decrease**. (Rule: successive % changes of +x and -x always give a net decrease of $x^2/100$.)

Profit & Loss

Q3.

A shopkeeper buys an item for ₹800 and sells it for ₹960. Find his profit percentage.

- (A) 15%
- (B) 18%
- (C) 20%
- (D) 25%

Answer: (C) 20%

Solution: Profit = $960 - 800 = 160$.
Profit% = $(160/800) \times 100 = 20\%$.

Profit & Loss

Q4.

A man sells an article at a loss of 15%. Had he sold it for ₹204 more, he would have gained 5%. Find the cost price of the article.

- (A) ₹900
- (B) ₹1000
- (C) ₹1020
- (D) ₹1100

Answer: (C) ₹1020

Solution: Let CP = x. SP at 15% loss = $0.85x$. SP at 5% gain = $1.05x$.
Difference = $1.05x - 0.85x = 0.20x = 204 \rightarrow x = 204/0.20 = \text{₹}1020$.

Ratio

Q5.

Two numbers are in the ratio 3:5. If 8 is added to each number, the ratio becomes 2:3. Find the two numbers.

- (A) 20 & 36
- (B) 24 & 40
- (C) 18 & 30
- (D) 21 & 35

Answer: (B) 24 & 40

Solution: Let numbers = $3x$ and $5x$.

$$(3x+8)/(5x+8) = 2/3 \rightarrow 9x + 24 = 10x + 16 \rightarrow x = 8.$$

Numbers = $3(8)=24$ and $5(8)=40$.

Ratio

Q6.

A sum of money is divided among A, B and C in the ratio 2:3:5. If C receives ₹900 more than A, find the total sum.

- (A) ₹2500
- (B) ₹2800
- (C) ₹3000
- (D) ₹3200

Answer: (C) ₹3000

Solution: Let shares = $2x$, $3x$, $5x$.

$$C - A = 5x - 2x = 3x = 900 \rightarrow x = 300.$$

$$\text{Total} = 2x+3x+5x = 10x = 10 \times 300 = \text{₹}3000.$$

Average

Q7.

The average of 5 numbers is 27. If one number is excluded, the average of the remaining 4 numbers becomes 25. Find the excluded number.

- (A) 30
- (B) 32
- (C) 35
- (D) 38

Answer: (C) 35

Solution: Sum of 5 numbers = $5 \times 27 = 135$.

Sum of remaining 4 numbers = $4 \times 25 = 100$.

$$\text{Excluded number} = 135 - 100 = \text{35}.$$

Average

Q8.

The average age of a class of 29 students is 15 years. If the teacher's age (45 years) is also included, find the new average age of all 30 people.

- (A) 15
- (B) 15.5
- (C) 16
- (D) 16.5

Answer: (C) 16

Solution: Sum of students' ages = $29 \times 15 = 435$.

Adding teacher: $435 + 45 = 480$.

$$\text{New average} = 480 / 30 = \text{16}.$$

Q9.

A can complete a piece of work in 12 days, and B can complete it in 15 days. If they work together, in how many days will the work be completed?

- (A) 6 days
- (B) $6\frac{2}{3}$ days
- (C) 7 days
- (D) 7.5 days

Answer: (B) $6\frac{2}{3}$ days (20/3 days)

Solution: A's 1-day work = $1/12$, B's 1-day work = $1/15$.

Together = $1/12 + 1/15 = (5+4)/60 = 9/60 = 3/20$.

Time = $20/3 = 6\frac{2}{3}$ days.

Q10.

A and B together can complete a piece of work in 10 days. A alone can complete it in 15 days. In how many days can B alone complete the work?

- (A) 20 days
- (B) 25 days
- (C) 30 days
- (D) 35 days

Answer: (C) 30 days

Solution: (A+B)'s 1-day work = $1/10$. A's 1-day work = $1/15$.

B's 1-day work = $1/10 - 1/15 = (3-2)/30 = 1/30$.

B alone takes **30 days**.

Q11.

A train 150 m long crosses a pole in 15 seconds. Find its speed in km/h.

- (A) 30 km/h
- (B) 33 km/h
- (C) 36 km/h
- (D) 40 km/h

Answer: (C) 36 km/h

Solution: Speed = Distance/Time = $150/15 = 10$ m/s.

Convert to km/h: $10 \times 18/5 = 36$ km/h.

Q12.

Two trains, 120 m and 180 m long, run on parallel tracks in opposite directions at 54 km/h and 36 km/h respectively. Find the time they take to completely cross each other.

- (A) 10 sec
- (B) 12 sec
- (C) 14 sec
- (D) 15 sec

Answer: (B) 12 sec

Solution: Relative speed (opposite directions) = $54+36 = 90$ km/h = $90 \times 5/18 = 25$ m/s.

Total distance to cover = $120+180 = 300$ m.

Time = $300/25 = 12$ seconds.

Simple Interest

Q13.

Find the Simple Interest on ₹5000 at 8% p.a. for 3 years.

- (A) ₹1000
- (B) ₹1100
- (C) ₹1200
- (D) ₹1300

Answer: (C) ₹1200

Solution: $SI = (P \times R \times T)/100 = (5000 \times 8 \times 3)/100 = \text{₹}1200$.

Compound Interest

Q14.

Find the Compound Interest on ₹10,000 at 10% p.a. for 2 years, compounded annually.

- (A) ₹2000
- (B) ₹2100
- (C) ₹2200
- (D) ₹2400

Answer: (B) ₹2100

Solution: Amount = $P(1+R/100)^2 = 10000 \times (1.1)^2 = 10000 \times 1.21 = 12,100$.
 $CI = 12,100 - 10,000 = \text{₹}2100$.

Probability

Q15.

A bag contains 4 red and 6 blue balls. One ball is drawn at random. Find the probability that it is red.

- (A) $1/2$
- (B) $2/5$
- (C) $3/5$
- (D) $1/3$

Answer: (B) $2/5$

Solution: Total balls = $4+6 = 10$. $P(\text{red}) = 4/10 = 2/5$.

Probability

Q16.

Two dice are thrown together. Find the probability of getting a sum of 7.

- (A) $1/6$
- (B) $1/9$
- (C) $1/12$
- (D) $5/36$

Answer: (A) $1/6$

Solution: Total outcomes = $6 \times 6 = 36$.
Favourable pairs summing to 7: $(1,6)(2,5)(3,4)(4,3)(5,2)(6,1) = 6$ outcomes.
 $P = 6/36 = 1/6$.

Number System

Q17.

Find the smallest 4-digit number exactly divisible by 12, 15 and 20.

- (A) 1000
- (B) 1020
- (C) 1040
- (D) 1060

Answer: (B) 1020

Solution: $LCM(12,15,20) = 60$.
Smallest 4-digit multiple of 60: $1000/60 \approx 16.67$, so next whole multiple is $17 \times 60 = 1020$.

Q18.**What is the sum of the first 20 natural numbers?**

- (A) 190
- (B) 200
- (C) 210
- (D) 220

Answer: (C) 210**Solution:** $\text{Sum} = n(n+1)/2 = 20 \times 21 / 2 = \mathbf{210}$.**Q19.****Find the HCF of 84 and 126.**

- (A) 21
- (B) 36
- (C) 42
- (D) 48

Answer: (C) 42**Solution:** $84 = 2^2 \times 3 \times 7$, $126 = 2 \times 3^2 \times 7$.Common factors: $2 \times 3 \times 7 = \mathbf{42}$.**Q20.****Find the LCM of 18, 24 and 36.**

- (A) 64
- (B) 72
- (C) 84
- (D) 96

Answer: (B) 72**Solution:** $18 = 2 \times 3^2$, $24 = 2^3 \times 3$, $36 = 2^2 \times 3^2$. $\text{LCM} = 2^3 \times 3^2 = 8 \times 9 = \mathbf{72}$.**Q21.****The present age of a father is 3 times that of his son. After 12 years, the father's age will be twice that of his son. Find their present ages.**

- (A) Father 30, Son 10
- (B) Father 33, Son 11
- (C) Father 36, Son 12
- (D) Father 39, Son 13

Answer: (C) Father 36, Son 12**Solution:** Let son = x, father = 3x. $3x+12 = 2(x+12) \rightarrow 3x+12 = 2x+24 \rightarrow x = 12$.Son = **12**, Father = $3 \times 12 = \mathbf{36}$.

Q22.

Five years ago, Sita was thrice as old as Gita. Ten years from now, Sita will be twice as old as Gita. Find Gita's present age.

- (A) 15
- (B) 18
- (C) 20
- (D) 25

Answer: (C) 20

Solution: Let Gita's present age = g, Sita's = s.

5 years ago: $s-5 = 3(g-5) \rightarrow s = 3g-10$.

10 years later: $s+10 = 2(g+10) \rightarrow$ substituting: $3g-10+10 = 2g+20 \rightarrow g = \mathbf{20}$.

(Check: Sita = 50; 5 yrs ago 45 = 3×15 ✓; 10 yrs later 60 = 2×30 ✓)

Q23.

Find the next number in the series: 2, 6, 12, 20, 30, ?

- (A) 36
- (B) 40
- (C) 42
- (D) 44

Answer: (C) 42

Solution: Differences: 4, 6, 8, 10, 12 (increasing by 2 each time).

$30 + 12 = \mathbf{42}$. (Pattern: each term = $n \times (n+1)$: $1 \times 2, 2 \times 3, 3 \times 4, 4 \times 5, 5 \times 6, 6 \times 7 = 42$)

Q24.

Find the next letter in the series: B, D, G, K, P, ?

- (A) T
- (B) U
- (C) V
- (D) W

Answer: (C) V

Solution: Gaps between letters increase by 1 each time: B(+2)D(+3)G(+4)K(+5)P(+6)→V.

B=2, D=4, G=7, K=11, P=16, next = $16+6 = 22 = \mathbf{V}$.

Q25.

In a certain code, 'CAT' is written as 'DBU'. How is 'DOG' written in that code?

- (A) EPH
- (B) EPG
- (C) FPH
- (D) EPI

Answer: (A) EPH

Solution: Each letter is shifted forward by 1: C→D, A→B, T→U.

Applying the same rule to DOG: D→E, O→P, G→H → **EPH**.

Q26.**If TABLE is coded as UBCMF, what is the code for CHAIR?**

- (A) DIBJS
- (B) DIBIS
- (C) DJBJS
- (D) DIBJR

Answer: (A) DIBJS

Solution: Each letter of TABLE is shifted forward by 1 to get UBCMF.
Applying to CHAIR: C→D, H→I, A→B, I→J, R→S → **DIBJS**.

Q27.**Statements: All pens are books. All books are tables. Conclusion: All pens are tables. Is this conclusion valid?**

- (A) Valid
- (B) Invalid
- (C) Cannot be determined
- (D) Partially valid

Answer: (A) Valid

Solution: This is a direct chain syllogism: pens $\subset$ books, books $\subset$ tables $\Rightarrow$ pens $\subset$ tables. The conclusion follows logically, so it is **valid**.

Q28.**Find the odd one out: Apple, Mango, Potato, Banana.**

- (A) Apple
- (B) Mango
- (C) Potato
- (D) Banana

Answer: (C) Potato

Solution: Apple, Mango and Banana are fruits; Potato is a vegetable — so **Potato** is the odd one out.

Q29.**Choose the correctly spelled word.**

- (A) Recieve
- (B) Receive
- (C) Receve
- (D) Receeve

Answer: (B) Receive

Solution: Rule: "i before e except after c" $\rightarrow$ since there is a 'c' before, it is "ei": **Receive**.

Q30.**Choose the word most similar in meaning (synonym) to 'Benevolent'.**

- (A) Cruel
- (B) Kind
- (C) Greedy
- (D) Arrogant

Answer: (B) Kind

Solution: 'Benevolent' means well-meaning and generous — closest in meaning to **Kind**.

Section 2 · Common + Job-Related Technical (20 Questions)

Covers JavaScript, React, HTML/CSS, PHP, MySQL/SQL, REST APIs, HTTP, Git, OOP and MVC — the core stack for the React Developer and PHP Developer roles. A few basic coding questions are included since it is not confirmed whether a separate coding round will be held.

JavaScript

T1.

What is the difference between var, let, and const in JavaScript?

Answer:

- **var** — function-scoped, can be re-declared and updated, is hoisted and initialized as `undefined`.
- **let** — block-scoped, can be updated but not re-declared in the same scope, hoisted but not initialized (temporal dead zone).
- **const** — block-scoped, cannot be reassigned after declaration (the binding is constant, though objects/arrays it holds can still be mutated internally).

In modern JavaScript, `let` and `const` are preferred over `var` because block scoping avoids many accidental-overwrite bugs.

```
var a = 1;      // function scoped
let b = 2;      // block scoped
const c = 3;    // block scoped, cannot reassign
if (true) {
  let b = 20;   // different 'b', only inside this block
}
```

JavaScript

T2.

What is a closure in JavaScript? Give an example.

Answer: A closure is created when a function "remembers" the variables from the scope in which it was defined, even after that outer function has finished executing. This lets inner functions access outer function variables.

```
function counter() {
  let count = 0;
  return function () {
    count++;
    return count;
  };
}
const increment = counter();
console.log(increment()); // 1
console.log(increment()); // 2 (count persists between calls)
```

Closures are commonly used for data privacy (private variables) and for functions like event handlers and callbacks that need to retain state.

JavaScript · Coding

T3.

Write a JavaScript function to check whether a given string is a palindrome.

Answer:

```
function isPalindrome(str) {
  const clean = str.toLowerCase().replace(/[^a-z0-9]/g, "");
  const reversed = clean.split("").reverse().join("");
  return clean === reversed;
}

console.log(isPalindrome("Madam")); // true
console.log(isPalindrome("Hello")); // false
```

Explanation: Convert to lowercase and strip non-alphanumeric characters, then compare the string with its reverse.

React

T4.

What is the difference between state and props in React?

Answer:

- **Props** ("properties") are passed from a parent component to a child component. They are **read-only** inside the child — the child cannot modify them.
 - **State** is data managed **within** a component itself, using `useState` (or `this.state` in class components). It can change over time and re-renders the component when updated.
- In short: props flow down from parent to child and are immutable to the child; state is local and mutable within the component that owns it.

T5.

Explain the `useState` and `useEffect` hooks with a short example.

Answer:

- **`useState`** lets a functional component hold and update local state.
- **`useEffect`** lets a component run "side effects" (API calls, subscriptions, DOM updates) after render, and optionally clean them up.

```
import { useState, useEffect } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Count: ${count}`;    // runs after every render where count changes
  }, [count]);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

The array `[count]` is the dependency list — the effect only re-runs when `count` changes.

T6.

What is the Virtual DOM, and how does React use it?

Answer: The Virtual DOM is a lightweight, in-memory JavaScript representation of the actual browser DOM. When a component's state or props change, React first updates this virtual copy, then compares ("diffs") it against the previous version using its reconciliation algorithm, and finally applies only the **minimum necessary changes** to the real DOM. This avoids costly full-page re-renders and makes UI updates fast and efficient.

T7.

What is the difference between a class selector and an id selector in CSS?

Answer:

- An **id** selector (`#header`) targets exactly one unique element on a page (an id should not repeat).
- A **class** selector (`.card`) can be applied to multiple elements, so it's reusable across many elements with shared styling.

In terms of CSS specificity, an id selector has higher priority (weight 100) than a class selector (weight 10), so id styles override class styles if both target the same element.

T8.

Explain the CSS box model.

Answer: Every HTML element is treated as a rectangular box made up of four layers, from inside out:

1. **Content** — the actual text/image.
2. **Padding** — space between content and border.
3. **Border** — the line around the padding.
4. **Margin** — space outside the border, separating the element from others.

By default (`box-sizing: content-box`), width/height apply only to content; with `box-sizing: border-box`, width/height include padding and border, which is usually easier to work with in layouts.

T9.

What is the difference between `include` and `require` in PHP?

Answer: Both are used to insert the contents of one PHP file into another. The difference is in error handling:

- **`include`** — if the file is not found, PHP raises a **warning** and the script **continues executing**.
- **`require`** — if the file is not found, PHP raises a **fatal error** and the script **stops immediately**.

`include_once` / `require_once` do the same thing but also prevent the same file from being included more than once. Use `require` for files essential to the app (e.g., a DB config), and `include` for optional ones (e.g., a footer).

T10.**How would you connect a PHP application to a MySQL database?**

Answer: This can be done using either the `mysqli` extension or `PDO` (PHP Data Objects). `PDO` is generally preferred because it supports multiple database drivers and works cleanly with prepared statements.

```
// Using PDO
try {
    $pdo = new PDO("mysql:host=localhost;dbname=radixweb_db", "root", "password");
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $stmt = $pdo->prepare("SELECT * FROM users WHERE email = :email");
    $stmt->execute(['email' => $email]);
    $user = $stmt->fetch(PDO::FETCH_ASSOC);
} catch (PDOException $e) {
    echo "Connection failed: " . $e->getMessage();
}
```

Using prepared statements (as above) also protects against SQL injection.

T11.**Write a PHP function to check whether a given number is prime.**

Answer:

```
function isPrime($num) {
    if ($num < 2) return false;
    for ($i = 2; $i <= sqrt($num); $i++) {
        if ($num % $i == 0) return false;
    }
    return true;
}

var_dump(isPrime(17)); // true
var_dump(isPrime(20)); // false
```

Explanation: A number is prime if it has no divisors other than 1 and itself. We only need to check divisors up to its square root for efficiency.

T12.**What is the difference between INNER JOIN and LEFT JOIN in SQL?**

Answer:

- **INNER JOIN** returns only the rows where there is a match in **both** tables.
- **LEFT JOIN** returns **all rows from the left table**, plus matching rows from the right table; if there's no match, the right table's columns are returned as `NULL`.

```
SELECT o.id, c.name
FROM orders o
LEFT JOIN customers c ON o.customer_id = c.id;
-- Returns every order, even if the matching customer record was deleted.
```

T13.**Write an SQL query to find the second-highest salary from an Employee table.**

Answer:

```
SELECT MAX(salary) AS second_highest_salary
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);

-- Alternative using LIMIT/OFFSET:
SELECT DISTINCT salary
FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

Explanation: The first query excludes the overall maximum, then finds the max of what's left. The second sorts salaries descending and skips the top row.

T14.**What is a REST API, and what are its key principles?**

Answer: REST (Representational State Transfer) is an architectural style for designing networked APIs, typically over HTTP. Key principles:

- **Stateless** — each request from client to server must contain all information needed; the server stores no client session state between requests.
- **Client-Server separation** — front end and back end evolve independently.
- **Resource-based URLs** — resources (e.g., `/users`, `/orders/5`) are identified by URLs, and operations use standard HTTP methods (GET, POST, PUT, DELETE).
- **Uniform interface** — consistent, predictable structure across endpoints.
- Responses are commonly returned in JSON format.

T15.**Explain common HTTP methods and status codes.**

Answer:

Methods:

- **GET** — retrieve data (no body change on server).
- **POST** — create a new resource.
- **PUT** — update/replace an existing resource fully.
- **PATCH** — partially update a resource.
- **DELETE** — remove a resource.

Common status codes:

- **200 OK** — request succeeded.
- **201 Created** — resource created successfully (typical for POST).
- **400 Bad Request** — invalid request from the client.
- **401 Unauthorized** — authentication required/failed.
- **404 Not Found** — resource doesn't exist.
- **500 Internal Server Error** — unexpected server-side error.

T16.**What is the difference between git merge and git rebase?**

Answer:

- **git merge** combines two branches by creating a new "merge commit" that ties both histories together. It preserves the full, true history of commits.
- **git rebase** moves/replays your branch's commits on top of another branch, producing a linear history without a merge commit — but it rewrites commit history, so it should be avoided on shared/public branches.

Rule of thumb: use `merge` for shared branches, and `rebase` to clean up your own local feature branch before merging.

T17.**Explain the four pillars of Object-Oriented Programming.**

Answer:

- **Encapsulation** — bundling data and methods together inside a class, and restricting direct access to internal data (via private/protected members).
- **Abstraction** — hiding complex implementation details and exposing only the necessary functionality (e.g., an interface or abstract class).
- **Inheritance** — a class (child) can reuse and extend the properties/methods of another class (parent).
- **Polymorphism** — the same method name behaves differently depending on the object calling it (e.g., method overriding in a subclass, or overloading).

T18.**What is the difference between an abstract class and an interface?**

Answer:

- An **abstract class** can have both fully implemented methods and abstract (unimplemented) methods; it can also hold state (properties). A class can inherit from only **one** abstract class.
- An **interface** only declares method signatures (a contract) with no implementation (in most languages), and a class can implement **multiple** interfaces.

Use an abstract class when classes share common code; use an interface when unrelated classes just need to guarantee the same set of behaviors.

T19.**Explain the MVC (Model-View-Controller) architecture pattern.**

Answer: MVC separates an application into three parts:

- **Model** — manages the data and business logic (e.g., interacting with the database).
- **View** — the presentation layer, what the user actually sees (HTML/UI).
- **Controller** — receives user input/requests, talks to the Model to fetch/update data, and decides which View to render.

This separation makes code more organized, testable, and easier to maintain — frameworks like Laravel (PHP) and many React-based architectures are inspired by or built around this pattern.

T20.**What is the difference between authentication and authorization? How would you implement basic login authentication?**

Answer:

- **Authentication** answers "Who are you?" — verifying identity (e.g., checking username/password at login).
- **Authorization** answers "What are you allowed to do?" — checking permissions after identity is confirmed (e.g., only an admin can delete a user).

A simple login flow: the user submits credentials → the server verifies the password (compared as a securely hashed value, e.g. using `password_hash()/password_verify()` in PHP, never plain text) → on success, the server issues a session or a token (like JWT) → that session/token is checked on each subsequent request to confirm the user is still authenticated, and their role is checked for authorization on protected actions.

Section 3 · Extra Interview Preparation (20 Questions)

A mix of technical follow-ups and HR/behavioural questions specific to this drive — including the stipend, probation and bond terms mentioned in the Radixweb notice. Sample answers are written in first person as a starting point; personalize them further before the interview.

HR

I1.

Tell me about yourself.

Sample answer: "I'm Vishal, currently pursuing my MCA at LJ University in Ahmedabad, after completing my BCA at JJKCC. Alongside my coursework, I'm a hands-on learner — I maintain personal repositories where I practice React, PHP/OOP concepts, and JavaScript, and I've built projects like CarZen, a Flutter and FastAPI/MySQL-based car marketplace app, and SecureVaultX, a secure file-storage project built on Spring Boot. I'm now looking to start my career as a Developer, and I'm applying for the React Developer and PHP Developer roles at Radixweb because they match the web development skills I've been building."

HR

I2.

Why do you want to work as a React Developer?

Sample answer: "I enjoy building user interfaces that feel fast and responsive, and React's component-based structure makes that easy to organize and reuse. I've practiced React through personal projects and repositories, and I like how hooks like useState and useEffect let you manage UI logic cleanly. I want to keep growing in frontend development, and React is currently one of the most in-demand libraries for that."

HR

I3.

Why do you want to work as a PHP Developer?

Sample answer: "PHP is where I first got comfortable with server-side and database-driven development — I've practiced OOP concepts in PHP and built small database-backed applications. I like that PHP powers a huge share of real-world web applications, and it pairs naturally with MySQL, which I'm also comfortable with. I see it as a strong, practical skill to build a backend career on."

HR

I4.

Why do you want to join Radixweb specifically?

Sample answer: "Radixweb has built a reputation as a software engineering partner that works across a wide range of technologies — from AI/ML to MERN to PHP — for startups and enterprises alike. As a fresher, that variety is appealing because it means I can keep learning across different kinds of projects. It's also based right here in Ahmedabad, so I can bring full focus and consistency to the role from day one."

HR

I5.

What are your key strengths?

Sample answer: "I'd say my biggest strength is that I'm a hands-on, self-driven learner — I don't just study concepts in class, I build small projects to actually apply them, which is how I ended up with several personal repositories and pinned GitHub projects. I'm also comfortable troubleshooting problems methodically, whether that's a tricky environment setup or a bug in my code, rather than giving up or guessing."

HR

I6.

What do you consider your biggest weakness?

Sample answer: "Since I mostly learn by building things on my own, I sometimes spend more time than needed exploring or perfecting a personal project instead of moving on. I've been working on this by setting clearer scopes and deadlines for myself before starting a task, which has helped me finish things faster without losing quality."

HR

I7.

Why should we hire you?

Sample answer: "I come in with practical, hands-on exposure to both frontend and backend development — not just theory — because I've consistently built and shipped my own projects alongside my MCA coursework. I'm quick to pick up a codebase, comfortable asking the right questions, and genuinely enjoy the process of building software, which means I'll keep improving quickly on the job."

HR · Drive-specific

I8.

Are you comfortable relocating to / working in Ahmedabad?

Sample answer: "Yes, absolutely — I'm already based in Ahmedabad since I'm currently pursuing my MCA at LJ University here, so there's no relocation needed on my end."

HR · Drive-specific

I9.

This role has a 6-month probation period with a stipend of ₹10,000 (₹5,000 paid monthly + ₹5,000 on successful completion). Are you comfortable with this structure?

Sample answer: "Yes, I understand that probation periods are common for freshers and are meant to be a mutual evaluation window. I see this as an opportunity to learn Radixweb's processes and prove myself, and I'm comfortable with the stipend structure during that period."
(Tip: it's fine to ask for clarity on exact numbers and the post-probation salary structure/date if it's not already explained in the drive.)

HR · Drive-specific

I10.

Are you okay with the 2-year service bond and the ₹1.5 Lakh security deposit requirement (via cheque)?

Sample answer: "Yes, I understand the bond is meant to protect the company's investment in training freshers, and I'm committed to building my career here for that period. I'd just like to fully understand the terms — for example, what happens to the deposit cheque if I complete the bond period, and the exact conditions under which it would be encashed — so I can be clear on the commitment before signing."
(Tip: it's completely reasonable and expected to ask for the bond agreement in writing before accepting an offer.)

HR

I11.

Where do you see yourself in the next 3-5 years?

Sample answer: "In the next few years, I want to grow from a fresher developer into someone who can independently own a feature or a small module end-to-end — understanding both the frontend and backend well enough to make good design decisions. I'd also like to mentor newer freshers the way I hope to be mentored when I join."

Project

I12.

Tell me about a project you have built.

Sample answer: "One project I worked on recently is CarZen, a car buying/selling marketplace app. It has an existing FastAPI and MySQL backend, and my part was building and polishing the Flutter frontend — screens like Browse Cars, Car Details, Sell/Add Car, Login/Register, Orders, and an Admin panel — making sure it was fully driven by real backend data rather than mock data, and responsive across mobile, tablet and web. It's also part of my MCA coursework as a group project. I've also built SecureVaultX, a personal project for secure file storage, using a proper three-tier architecture (Spring Boot backend, Flutter frontend) to practice clean backend design and encryption concepts. While these used Flutter and FastAPI/Spring Boot rather than React or PHP directly, the underlying skills — consuming REST APIs, structuring an app cleanly, and working with relational databases — carry over directly, and I've practiced React and PHP OOP separately in my own repositories."

Project

I13.

What was the most challenging part of that project, and how did you solve it?

Sample answer: "With CarZen, the biggest challenge was making sure the frontend only ever reflected what the backend actually supported — I had to treat the backend source code and API documentation as the single source of truth rather than guessing endpoint names or response fields, and work incrementally instead of rewriting large parts of the app at once. It taught me to read existing code carefully before changing anything, and to check my assumptions against the real API rather than assuming how it should work."

React · Technical

I14.

What are controlled and uncontrolled components in React?

Answer: In a **controlled component**, form data (like an input's value) is held in React state, and every change goes through an `onChange` handler that updates that state — React fully controls the input's value. In an **uncontrolled component**, the DOM itself manages the input's value, and React reads it only when needed (e.g., via a `ref`), rather than tracking every keystroke in state. Controlled components are generally preferred for predictable validation and state management.

I15.**How would you prevent SQL injection in a PHP application?**

Answer: The main defense is to **never concatenate user input directly into a SQL query**. Instead, use **prepared statements** with bound parameters (via PDO or mysqli), so user input is always treated as data, never as executable SQL.

```
// Vulnerable:
$query = "SELECT * FROM users WHERE email = '$email'";

// Safe (PDO prepared statement):
$stmt = $pdo->prepare("SELECT * FROM users WHERE email = :email");
$stmt->execute(['email' => $email]);
```

Additional good practice: validate/sanitize input, use least-privilege database accounts, and avoid displaying raw database errors to users.

I16.**What is normalization in a database? Explain briefly with an example.**

Answer: Normalization is the process of organizing data to reduce redundancy and avoid update anomalies, typically by splitting data into related tables. For example, instead of storing a customer's full address as repeated text in every order row, you'd store customers in a separate `customers` table and reference them from `orders` via a `customer_id` foreign key. This way, updating a customer's address only requires one change, not one per order. Common normal forms are 1NF, 2NF and 3NF, each removing a specific kind of redundancy.

I17.**What is the difference between == and === in JavaScript?**

Answer: `==` is the "loose equality" operator — it compares two values **after converting them to the same type** if they differ (type coercion). `===` is "strict equality" — it compares both **value and type**, with no conversion.

```
"5" == 5    // true  (string is converted to number)
"5" === 5    // false (different types: string vs number)
null == undefined // true
null === undefined // false
```

Best practice: use `===` by default to avoid unexpected type-coercion bugs.

I18.**Write a function to find the largest number in an array without using a built-in max function.**

Answer (JavaScript):

```
function findLargest(arr) {
  let largest = arr[0];
  for (let i = 1; i < arr.length; i++) {
    if (arr[i] > largest) {
      largest = arr[i];
    }
  }
  return largest;
}

console.log(findLargest([3, 41, 7, 19, 25])); // 41
```

Explanation: Start by assuming the first element is the largest, then loop through the rest, updating whenever a bigger value is found.

I19.**Have you implemented login/authentication in any of your own projects? How did you approach it?**

Sample answer: "Yes — in CarZen, I worked with an existing authentication service and token-based login flow, making sure the app only cleared a stored token on real authentication failures (like a 401/403 response) and not on unrelated network errors, so users wouldn't get logged out unnecessarily. In SecureVaultX, a personal project, I used Spring Security with session-based authentication for a more secure, three-tier backend setup. These were learning-focused projects to understand authentication flows properly — I wouldn't call myself a security expert, but I'm comfortable with the fundamentals of how login, sessions and protected routes work."

I20.**How do you handle tight deadlines or work pressure?**

Sample answer: "I try to break a task down into smaller, clear steps as early as possible, and tackle the riskiest or most uncertain part first rather than saving it for last. I also believe in asking questions early instead of getting stuck silently — I'd rather clarify something with a teammate or lead early on than submit something incorrect at the deadline."

Bonus Tip

Before the interview, prepare 2-3 genuine questions to ask the interviewer (e.g., "What does the onboarding/training look like for freshers during the 6-month probation?" or "What kind of projects does the React/PHP team typically work on?"). Asking thoughtful questions signals genuine interest and is almost always well received.